Experiment No. 5

Anomaly Detection in IoT Sensor Data Using Unsupervised Learning

1. Aim

To understand and implement unsupervised Machine Learning techniques for detecting anomalies (faults) in IoT sensor data, evaluate their performance using standard metrics, and understand why lightweight models matter for resource-constrained IoT devices — all done entirely in software, without physical hardware.

2. Objectives

- Understand what an "anomaly" or "fault" means in an IoT system, and why supervised learning usually cannot be used to detect it.
- Simulate a realistic industrial IoT sensor dataset (an electric motor with temperature, vibration, current, and RPM sensors) with a small number of injected faults.
- Implement a simple statistical anomaly detection method (Z-score thresholding) from first principles.
- Implement an unsupervised Machine Learning model (Isolation Forest) for anomaly detection and compare it with the statistical method.
- Evaluate an anomaly detector using Precision, Recall, F1-score, and a Confusion Matrix.
- Understand what makes a model "lightweight" and why this matters for deployment on resource-constrained IoT devices (microcontrollers, edge sensors).

3. Software Requirements

No hardware or sensor kit is required — everything is simulated in software.

- Python 3.8 or above
- Google Colab / Jupyter Notebook / VS Code
- Python libraries: numpy, pandas, matplotlib, scikit-learn

4. Theory

4.1 What is Anomaly / Fault Detection in IoT?

An IoT system typically has sensors continuously reporting readings from a machine, device, or environment — e.g. a motor's temperature, vibration, and current draw. Most readings stay within a normal, expected range. An anomaly (fault or outlier) is a reading, or pattern of readings, that deviates significantly from this normal behaviour — for example a sudden temperature spike (possible overheating), unusual vibration (possible bearing wear), or abnormal current draw (possible overload). Detecting these early allows a system to raise an alert or shut down safely before real damage occurs — known as predictive maintenance.

4.2 Why Unsupervised Learning, not Supervised Learning?

Anomaly detection is usually done with unsupervised learning because:

- Faults are rare — a machine runs normally 95–99% of the time, so there are rarely enough labelled faulty examples to train a supervised classifier properly.

- New, previously unseen fault types can occur that a supervised model was never trained to recognise.
- Manually labelling every historical sensor reading as normal/faulty is expensive and requires an expert.

The model is therefore trained only on sensor readings (mostly normal), with no label given during training; it learns what "normal" looks like and flags anything that does not fit as a possible anomaly. The dataset's true_anomaly column is used only afterwards to check performance, never during detection itself — exactly as in a real deployment.

4.3 Lightweight ML Models for Resource-Constrained IoT Devices

- Large deep learning models need significant memory/compute and are usually unsuitable for small microcontrollers.
- Lightweight models — simple statistical rules, small decision trees, or compact ensembles like Isolation Forest — need very little memory and are fast, so they can run on constrained hardware.
- Common pattern: train on a powerful machine using historical data, then deploy only the small trained model (or a few numeric thresholds) to the edge device.
- Trade-off: lightweight models are cheap to run but may be less accurate/flexible than larger models.

Both methods in this practical were chosen because they are lightweight: Z-score needs only a mean and standard deviation per sensor, and Isolation Forest uses shallow trees rather than a large neural network.

4.4 Quick Reference

Method	How it decides "anomaly"	Resource cost
Z-score thresholding	Reading is too many standard deviations from the mean	Extremely low
Isolation Forest	Reading is unusually easy to "isolate" from the rest of the data	Low

5. The Simulated IoT Dataset

An electric motor fitted with 4 sensors, reporting 500 readings over time:

Column	Sensor / meaning	Normal range (approx.)
temperature_C	Motor casing temperature sensor	~51–69 °C
vibration_mms	Vibration sensor (accelerometer)	~1.2–2.8 mm/s
current_A	Current draw sensor (clamp meter / CT sensor)	~8–12 A
rpm	Rotational speed sensor (tachometer)	~1440–1560 RPM
true_anomaly	Hidden answer key (1 = fault secretly injected), used only for evaluation	0 or 1

500 readings were generated, of which 25 (5%) have a randomly injected fault pattern (mild overheating, bearing wear, or overload) layered on top of otherwise normal sensor values — a subtle drift rather than an obviously extreme spike.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(7)
n = 500
temperature = np.random.normal(60, 3, n)   # deg C
vibration   = np.random.normal(2.0, 0.4, n) # mm/s
current     = np.random.normal(10, 1, n)    # Amps
rpm         = np.random.normal(1500, 30, n) # RPM
true_anomaly = np.zeros(n, dtype=int)

rng = np.random.default_rng(7)
anomaly_idx = rng.choice(n, size=25, replace=False)  # 5% of readings
for idx in anomaly_idx:
    fault_type = rng.integers(0, 3)
    if fault_type == 0:                      # mild overheating
        temperature[idx] += rng.uniform(8, 15)
        vibration[idx]   += rng.uniform(0.5, 1.0)
    elif fault_type == 1:                    # bearing wear
        vibration[idx] += rng.uniform(1.5, 3.0)
        current[idx]   += rng.uniform(1.0, 2.0)
    else:                                     # overload
        current[idx] += rng.uniform(3, 5)
        rpm[idx]     -= rng.uniform(60, 120)
    true_anomaly[idx] = 1

df = pd.DataFrame({
    "reading_id": np.arange(1, n+1),
    "temperature_C": temperature.round(2),
    "vibration_mms": vibration.round(2),
    "current_A": current.round(2),
    "rpm": rpm.round(1),
    "true_anomaly": true_anomaly
})
df.to_csv("motor_sensor_dataset.csv", index=False)
df.head()
```

Note: in a real deployment this data would be logged from actual sensors over days/weeks, and the true_anomaly column would not be known in advance — it exists here purely as a teaching aid to measure how good the unsupervised methods are.

PART A — Method 1: Z-score Thresholding

What is a Z-score, and how does this method work?

A Z-score tells you how many standard deviations a value is away from the mean of its column: z = (value − mean) / standard_deviation. For a normal, well-behaved sensor, most values sit close to the mean, so most Z-scores are small. A large |Z-score| means that reading is unusually far from what's typical for that sensor. Here, a row is flagged as anomalous if any one of its sensors has |Z-score| greater than a chosen threshold — commonly 3, which for roughly bell-shaped data means fewer than about 0.3% of purely normal readings would be flagged by chance alone. This method needs nothing but the mean

and standard deviation of each sensor — exactly the kind of lightweight computation that can run on even the smallest microcontroller.

```
import pandas as pd, numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

df = pd.read_csv("motor_sensor_dataset.csv")
sensor_cols = ["temperature_C", "vibration_mms", "current_A", "rpm"]

# Z-score = (value - mean) / std, computed per sensor column
z_scores = (df[sensor_cols] - df[sensor_cols].mean()) / df[sensor_cols].std()
threshold = 3

# Flag a row if ANY sensor's |z-score| crosses the threshold
z_pred = (np.abs(z_scores) > threshold).any(axis=1).astype(int)
df["zscore_anomaly"] = z_pred

print("Flagged anomalies:", z_pred.sum())
print("Precision:", round(precision_score(df['true_anomaly'], z_pred), 3))
print("Recall:", round(recall_score(df['true_anomaly'], z_pred), 3))
print("F1-score:", round(f1_score(df['true_anomaly'], z_pred), 3))
```

Output — Z-score Method

```
Flagged anomalies: 22
Precision: 0.909
Recall: 0.8
F1-score: 0.851
```

How to read this: the method flagged 22 readings as anomalies. Precision of 0.909 means 90.9% of the readings it flagged were genuinely faulty (only a couple of false alarms). Recall of 0.8 means it caught 80% of the 25 true faults — it missed a few, most likely ones where only one sensor drifted slightly, not enough to cross the threshold on its own.

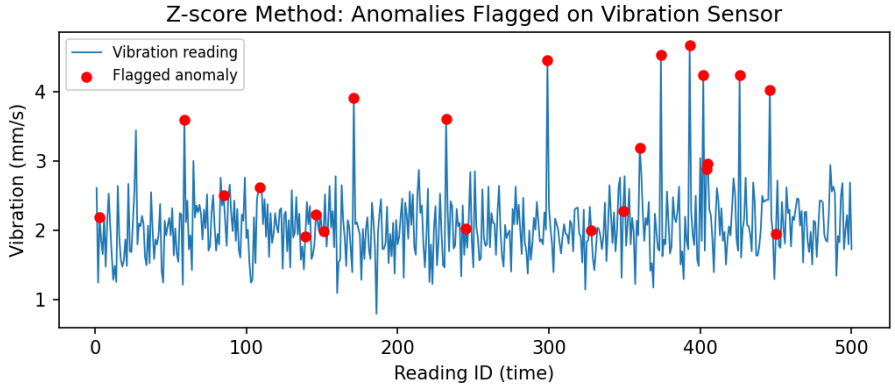


Fig. A.1 — Vibration sensor over time; red points are readings flagged as anomalies by the Z-score method.

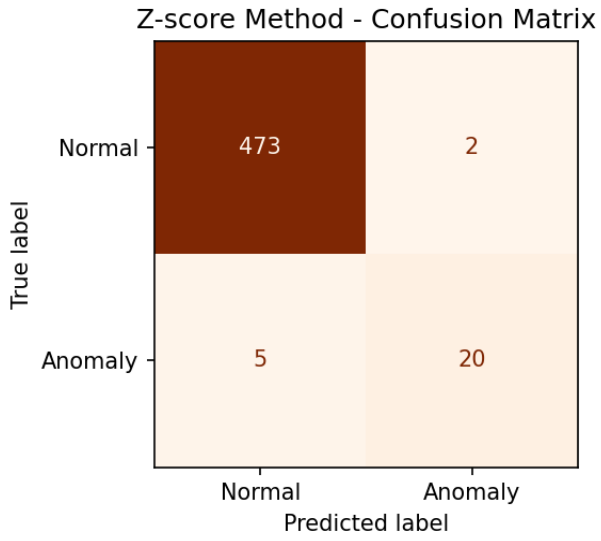


Fig. A.2 — Z-score method confusion matrix.

PART B — Method 2: Isolation Forest (Unsupervised ML)

What is Isolation Forest actually doing?

Isolation Forest works on the idea that anomalies are "few and different", so they should be easier to isolate (separate from the rest of the data) than normal points. To isolate a point, the algorithm builds a tree by repeatedly picking a random sensor and a random split value between that sensor's min and max, dividing the data in two, and repeating recursively — similar in structure to a Decision Tree, except the split feature and value are chosen randomly rather than by which split is "best".

A normal point sitting in a dense region typically takes many random splits to separate from everything else. An anomaly, sitting far from the crowd, usually gets isolated in just a few splits, since there is a lot of "empty space" around it that a random split is likely to land in. Isolation Forest builds many such random trees and, for every point, measures the average number of splits (path length) needed to isolate it. Points with a short average path length are scored as more anomalous.

Unlike the Z-score method, Isolation Forest looks at all sensors together, so it can catch a "joint" anomaly — e.g. a reading where no single sensor is far outside its own normal range, but the particular combination of vibration + current together is unusual — something the simple per-sensor Z-score check could miss.

```
from sklearn.ensemble import IsolationForest

X = df[sensor_cols]

# contamination = our estimate of what fraction of readings are anomalies
iso = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
iso.fit(X)                        # learns what "normal" looks like
raw_pred = iso.predict(X)         # -1 = anomaly, 1 = normal
iso_pred = (raw_pred == -1).astype(int)
df["iso_anomaly"] = iso_pred

print("Flagged anomalies:", iso_pred.sum())
print("Precision:", round(precision_score(df['true_anomaly'], iso_pred), 3))
```

```
print("Recall:", round(recall_score(df['true_anomaly'], iso_pred), 3))
print("F1-score:", round(f1_score(df['true_anomaly'], iso_pred), 3))
```

Output — Isolation Forest (contamination = 0.05)

```
Flagged anomalies: 25
Precision: 0.8
Recall: 0.8
F1-score: 0.8
```

How to read this: Isolation Forest flagged exactly 25 readings (because contamination=0.05, i.e. "expect about 5% of the data to be anomalous", and 5% of 500 is 25). It achieved 80% recall, catching the same overall proportion of true faults as the Z-score method, but with a few different false alarms — its precision (0.8) is slightly lower here because it flagged 5 normal readings that happened to look somewhat unusual across the combination of sensors.

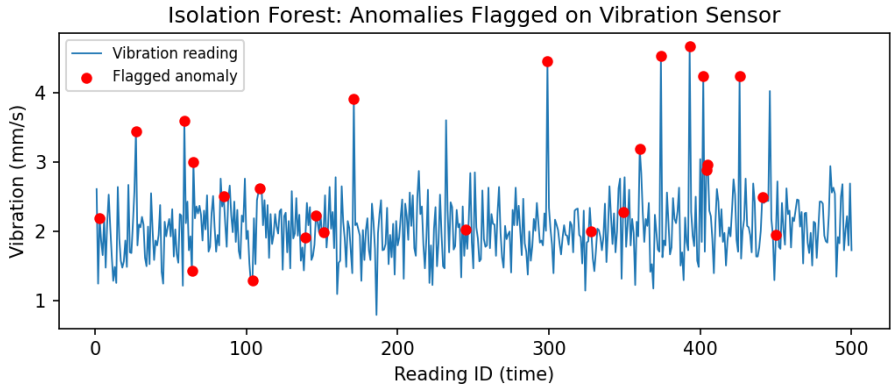


Fig. B.1 — Vibration sensor over time; red points are readings flagged as anomalies by Isolation Forest.

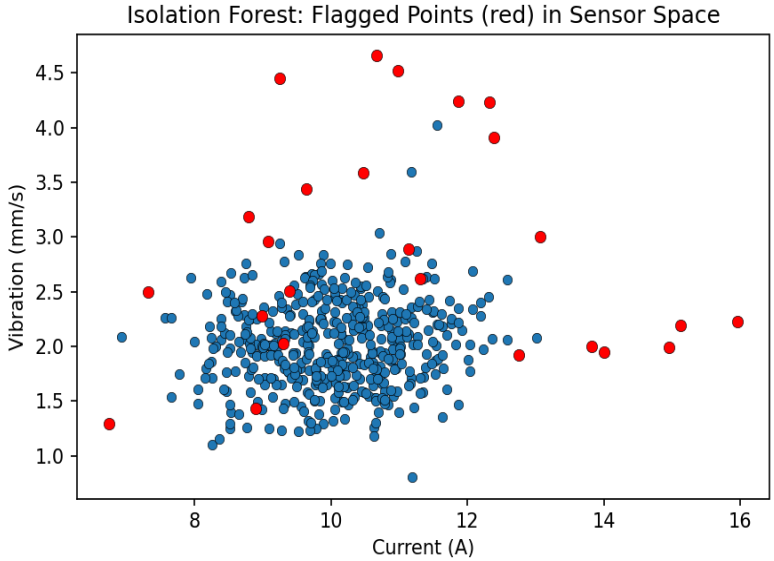


Fig. B.2 — Current vs Vibration sensor space; red points are flagged anomalies, sitting away from the main cluster.

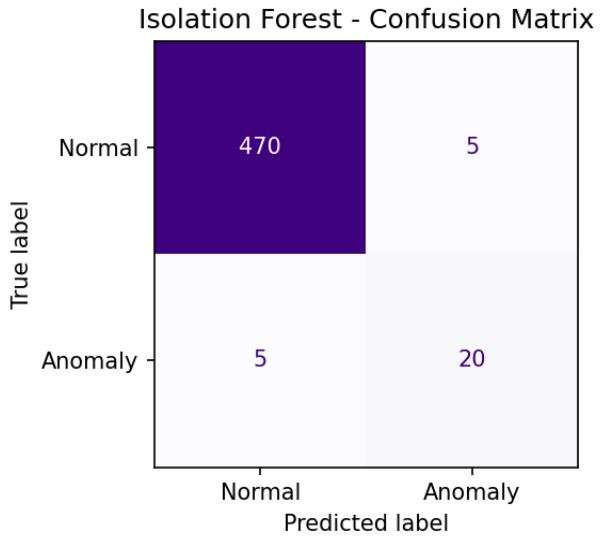


Fig. B.3 — Isolation Forest confusion matrix.

6. Understanding the Evaluation Metrics

Both methods were judged with the same three metrics. Since the true_anomaly answer key is secretly known, each prediction can be compared against it and sorted into one of 4 outcomes — the same idea as a confusion matrix, applied to "Normal" vs "Anomaly":

- True Positive (TP) — actually a fault, correctly flagged as anomaly.
- False Positive (FP) — actually normal, wrongly flagged as anomaly (a false alarm).
- False Negative (FN) — actually a fault, wrongly left unflagged (a missed fault — often the most costly mistake in a real machine).
- True Negative (TN) — actually normal, correctly left unflagged.

From these 4 counts:

```
Precision = TP / (TP + FP)   ->  "Of all the alarms raised, how many were real faults?"
Recall    = TP / (TP + FN)   ->  "Of all the real faults, how many did we catch?"
F1-score  = 2 x (Precision x Recall) / (Precision + Recall)  ->  a single balanced score
```

Why not just use accuracy? Because anomalies are rare (25 out of 500 readings, i.e. 5%), a lazy model that predicts "Normal" for everything would already be 95% accurate — while being completely useless, since it would catch zero faults. This is why anomaly detection is evaluated with Precision, Recall, and F1-score instead of plain accuracy: they specifically measure how well the rare, important "anomaly" class is handled.

Note: Precision and Recall usually trade off against each other. Raising the Z-score threshold (e.g. from 3 to 4) would flag fewer readings, increasing precision but lowering recall — and vice versa. There is no single "correct" threshold; it depends on how costly a missed fault is versus a false alarm for the specific IoT application.

7. Summary of Results

Method	Flagged	Precision	Recall	F1-score
Z-score Thresholding	22	0.909	0.8	0.851
Isolation Forest	25	0.8	0.8	0.8

Both lightweight methods caught 80% of the injected faults. The Z-score method had fewer false alarms here (higher precision) because the simulated faults mostly involved at least one sensor drifting clearly outside its normal range; Isolation Forest's advantage would become more apparent with faults only detectable by looking at multiple sensors together, since it considers all 4 sensors jointly rather than one at a time.

8. Conclusion

In this practical, IoT sensor data from a simulated electric motor was used to detect faults using two unsupervised, lightweight methods: a simple statistical Z-score threshold, and an unsupervised Machine Learning model, Isolation Forest. Both were trained without ever being shown which readings were actually faulty — exactly as would be the case in a real deployment. Their performance was measured using Precision, Recall, and F1-score, which are far more meaningful than plain accuracy for rare-event problems like fault detection. Because both methods need very little memory and computation, either could realistically run directly on a resource-constrained IoT edge device, making them practical choices for real-world predictive maintenance systems.

9. Exercise for Students

Change the Isolation Forest's contamination parameter from 0.05 to 0.10, re-run the model, and print its new Precision, Recall, and F1-score.

```
iso_ex = IsolationForest(n_estimators=100, contamination=0.10, random_state=42)
iso_ex.fit(X)
raw_pred_ex = iso_ex.predict(X)
iso_pred_ex = (raw_pred_ex == -1).astype(int)

print("Flagged anomalies:", iso_pred_ex.sum())
print("Precision:", round(precision_score(df['true_anomaly'], iso_pred_ex), 3))
print("Recall:", round(recall_score(df['true_anomaly'], iso_pred_ex), 3))
print("F1-score:", round(f1_score(df['true_anomaly'], iso_pred_ex), 3))
```

```
Flagged anomalies: 50
Precision: 0.46
Recall: 0.92
F1-score: 0.613
```